10주차 과제 · AI 도구를 활용한 문제해결

리눅스 VM 3-Tier 로그인 서비스 구축

Web(Nginx) - WAS(Gunicorn·Flask) - DB(MariaDB) 구성 및 호스트 OS 브라우저 ID/PW 로그인

이름	(기입)	학번	(기입)
작성일	2026-11-05	AI 도구	Claude Code (Anthropic)
결과물	GitHub `th0079/system` 저장소의 `week10/` 디렉터리 (코드·설정·자동 설치 스크립트·테스트)		

0. 과제 개요와 결과 요약

문제 — 리눅스 가상머신에 3-Tier 아키텍처 환경을 구성하고, 호스트 운영체제에서 웹 브라우저로 접속해 ID/PW로 로그인하는 서비스를 개발한다.

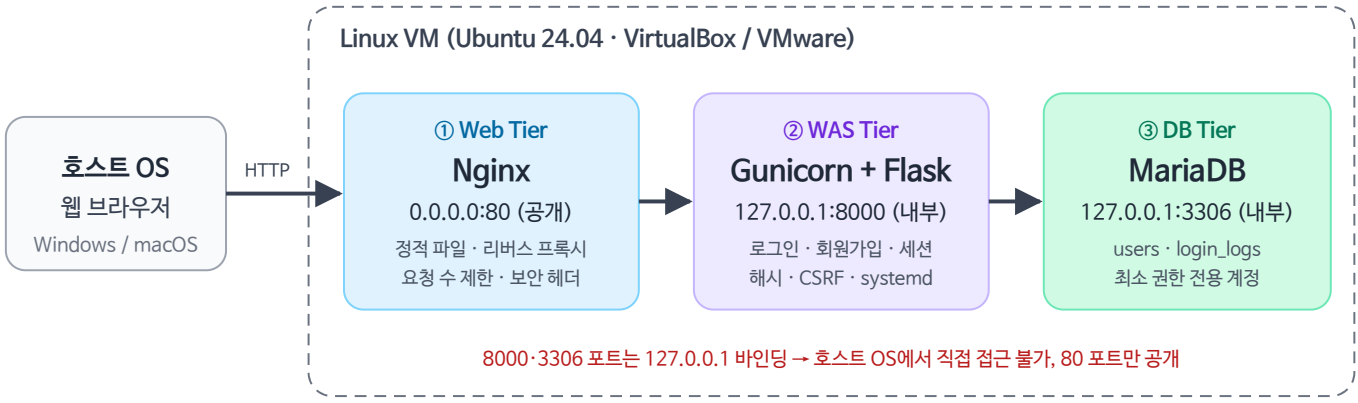


그림 1. 구성한 3-Tier 아키텍처 (한 대의 VM 안에서 포트·바인딩·OS 계정으로 계층 분리)

계층	구성 요소	역할	접근 범위
Web Presentation	Nginx 1.24	정적 파일(CSS) 직접 응답, 동적 요청을 WAS로 프록시, 로그인 요청 수 제한, WAS 장애 시 안내 페이지	`0.0.0.0:80`
WAS Application	Gunicorn + Flask (Python)	로그인·로그아웃·회원가입, 세션 관리, 비밀번호 해시(scrypt), CSRF 검증, systemd 서비스	`127.0.0.1:8000`
DB Data	MariaDB 10.11	회원(`users`)·로그인 이력(`login_logs`) 저장, SELECT/INSERT/UPDATE 권한만 가진 전용 계정	`127.0.0.1:3306`

결과 — AI 도구가 작성한 자동 설치 스크립트(`deploy/setup.sh`)를 Ubuntu 24.04 + systemd 환경에서 실행해 3계층을 구성했고, E2E 테스트 **28개 항목 모두 통과**, 다른 IP에서 접속 시 80 포트만 열리고 8000·3306은 거부됨, 재시작 후 세 서비스가 자동으로 올라오는 것까지 확인했다.
본인 VM 적용 결과: (하이퍼바이저/네트워크 방식, 접속 주소, 성공 여부 기입 — 예: VirtualBox NAT + 포트포워딩, http://localhost:8080 접속 성공)

사용한 AI 도구와 설정값

항목	값
AI 도구	**Claude Code** (Anthropic) — 대화형 챗봇이 아니라, 명령 실행·파일 작성·테스트까지 직접 수행하는 **에이전트형** 코딩 도구
사용 방식	Claude 데스크톱 앱 → 클라우드 세션. GitHub 저장소 `th0079/system` 연동, 작업 브랜치에 커밋·푸시
모델	세션 기본 모델 사용 (직접 변경하지 않음) · 모델명: (기입)
추론 노력(effort)	시작 시 `max` → 작업 도중 사용자가 변경
권한 모드	`auto` (AI가 명령 실행·파일 수정을 자동 승인으로 진행)
실행 환경	Anthropic 관리형 클라우드 컨테이너 (Ubuntu 24.04 LTS, Claude Code 2.1.318)
기타	자동 PR 생성 꺼짐, 웹 검색 사용 안 함

1. AI 도구 사용 전 예상 질문 리스트

문제를 읽고 AI 도구를 쓰기 전에, 해결 과정에서 물어봐야 할 것으로 예상한 질문 (총 15개)

A. 환경·설계

1. VirtualBox(또는 VMware)에 Ubuntu Server를 설치할 때 권장 사양과 네트워크 설정은?
2. 3-Tier 아키텍처란 무엇이고 Web 서버, WAS, DB는 각각 어떤 역할을 하나?
3. VM 한 대에 3계층을 모두 올려도 되나? 각 계층에 어떤 소프트웨어를 쓰는 것이 좋나?

B. 설치·연동

4. Nginx(또는 Apache) 설치와 실행 방법은?
5. WAS로 무엇을 쓸까(Tomcat, Flask, Node.js)? 설치 방법은?
6. MySQL/MariaDB 설치, root 비밀번호 설정, DB·계정 생성 방법은?
7. Nginx와 WAS를 연동하는(리버스 프록시) 설정은?

C. 로그인 서비스 개발

8. 로그인 페이지(HTML)와 로그인 처리 코드는 어떻게 작성하나?
9. 사용자 테이블은 어떻게 설계하고, 비밀번호는 어떻게 저장하나?
10. 로그인 상태를 유지하는 방법(세션/쿠키)은?
11. WAS에서 DB에 접속하는 코드는?

D. 접속·문제 해결

12. 호스트 OS 브라우저에서 VM 웹 서버에 접속하려면 네트워크(NAT 포트포워딩/브리지/호스트 전용)를 어떻게 설정하나?
13. 접속이 안 될 때 방화벽(ufw/firewalld) 설정은?
14. 502 Bad Gateway가 뜨면 어떻게 해결하나?
15. VM을 재부팅해도 서비스가 자동으로 실행되게 하려면?

2. AI 도구로 질문·답변하며 문제 해결 (실제 적용한 부분만 정리)

2-1. 실제로 입력한 질문

#	입력 내용	비고
Q1	과제 안내문 전체(문제 + 과제 정리 방법)를 그대로 붙여 넣음	첫 전송을 중단하고 같은 내용을 한 번 더 전송
—	"노력 변경" (작업 도중 추론 노력 설정을 바꿨다는 알림)	질문이 아니라 설정 변경

질문은 사실상 **1개**였다. 에이전트형 도구라서 AI가 문제를 스스로 아래 하위 작업으로 나누고, 명령을 실행해 보며 답을 만들었다. 아래는 그 답변 중 **실제 결과물에 적용된 부분**만 추린 것이다.

A-1. 아키텍처·기술 스택 결정 설계

VM 한 대 안에서 **포트·바인딩 주소·OS 계정**으로 계층을 나눔. Web=Nginx, WAS=Gunicorn+Flask(Python이 Ubuntu에 기본 설치되어 있어 준비가 간단), DB=MariaDB. 호스트에는 80 포트만 공개하고 WAS·DB는 `127.0.0.1`에만 바인딩.

적용: 그림 1의 구조 그대로 구현. 디렉터리도 `web/ · app/ · db/`로 계층별로 나눔.

A-2. DB Tier — 스키마, 전용 계정, 외부 접속 차단 DB

```
-- db/schema.sql (일부)
CREATE TABLE users (
  id INT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(20) NOT NULL UNIQUE,
  password_hash VARCHAR(255) NOT NULL,   -- 평문 대신 해시 저장
  name VARCHAR(30) NOT NULL, created_at DATETIME DEFAULT CURRENT_TIMESTAMP, last_login_at DATETIME NULL);
-- setup.sh : WAS 전용 계정에 필요한 권한만 부여 (최소 권한)
GRANT SELECT, INSERT, UPDATE ON loginapp.* TO 'loginapp'@'127.0.0.1';
# db/60-loginapp.cnf
[mysqld]
bind-address = 127.0.0.1
```

적용: 로그인 시도를 기록하는 `login_logs` 테이블 추가. MariaDB는 `'localhost'`(소켓)와 `'127.0.0.1'`(TCP)을 다른 호스트로 보므로 계정을 두 개 다 만듦 — 혼자였다면 몰랐을 부분.

A-3. WAS Tier — 로그인 처리 코드 WAS

```
# app/app.py (핵심 부분)
user = query_one("SELECT id, username, password_hash, name FROM users WHERE username = %s", (username,))
password_ok = check_password_hash(user["password_hash"] if user else DUMMY_HASH, password)
if not (user and password_ok):
    flash("아이디 또는 비밀번호가 올바르지 않습니다.", "error")   # ID/PW 중 무엇이 틀렸는지 숨김
    return render_template("login.html", username=username), 401
session.clear()                       # 로그인 시 세션 새로 발급
session["user_id"] = user["id"]
```

- **SQL Injection 방지**: 문자열을 이어 붙이지 않고 `%s` 자리표시자로 값 전달
- **비밀번호 해시**: `generate_password_hash`(scrypt) 로 저장, `check_password_hash` 로 비교
- **세션**: Flask 서명 쿠키, `HttpOnly`·`SameSite=Lax`, 30분 유지
- **CSRF 토큰**: 모든 POST 폼에 숨김 필드로 넣고 서버에서 비교

적용: 위 코드 + 회원가입(입력값 검증, 중복 ID 409), 로그아웃(POST), `/health` 헬스체크, 메인 화면에 3계층 정보 표시.

A-4. WAS 서비스화 — Gunicorn + systemd WAS

```
# app/gunicorn.conf.py
bind = "127.0.0.1:8000"     # 외부에서 직접 접근 불가
workers = 2
# deploy/loginapp.service (일부)
[Service]
User=loginapp               # root 가 아닌 전용 계정으로 실행
ExecStart=/opt/loginapp/venv/bin/gunicorn --config /opt/loginapp/app/gunicorn.conf.py app:app
Restart=on-failure
[Install]
WantedBy=multi-user.target   # 부팅 시 자동 시작
```

적용: 예상 질문 15번(재부팅 후 자동 실행)의 답. 컨테이너를 재시작해 nginx·mariadb·loginapp 모두 `active` 확인.

A-5. Web Tier — Nginx 리버스 프록시 Web

```
# web/nginx-loginapp.conf (일부)
upstream loginapp_was { server 127.0.0.1:8000; }
server {
    listen 80 default_server;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  # 실제 접속자 IP 전달
    location /static/ { alias /var/www/loginapp/static/; }       # 정적 파일은 Nginx 가 직접
    location = /login { limit_req zone=loginapp_login burst=30 nodelay; proxy_pass http://loginapp_was; }
    location / { proxy_pass http://loginapp_was; }
    error_page 502 503 504 /50x.html;                            # WAS 장애 안내
}
```

적용: 그대로 적용. WAS를 일부러 끄면 Nginx가 직접 안내 페이지(502)를 보여주고, CSS는 계속 200으로 응답 → 계층이 분리되어 있음을 눈으로 확인.

A-6. 호스트 OS에서 접속 — 네트워크·방화벽 접속

VM 네트워크	설정	브라우저 주소
VirtualBox NAT(기본)	포트 포워딩: 호스트 `127.0.0.1:8080` → 게스트 `80`	`http://localhost:8080`
VirtualBox 호스트 전용	어댑터2 추가 후 `ip -4 addr` 로 IP 확인	`http://192.168.56.x`
VMware NAT / 브리지	추가 설정 없음	`http://<VM IP>`

적용: `setup.sh` 가 ufw/firewalld가 켜져 있을 때만 80/tcp를 열고, Rocky Linux면 SELinux 설정(`httpd_can_network_connect`)도 처리. 설치가 끝나면 접속 주소를 출력.

A-7. 자동 설치 스크립트 + 검증 운영

```
sudo bash week10/deploy/setup.sh          # 패키지 → DB → WAS → Web → 방화벽 → 데모 계정 (7단계)
sudo bash week10/tests/e2e_test.sh        # curl 로 28개 항목 자동 점검
```

적용: 여러 번 실행해도 안전(비밀번호·데이터 유지). DB·SECRET_KEY 비밀번호는 무작위 생성해 `/etc/loginapp/loginapp.env` (권한 640)에 저장.

2-2. 진행 중 실제로 부딪힌 문제와 AI의 대처

문제	원인	대처(적용)
AI 작업 환경에 systemd가 없음	컨테이너는 PID 1이 systemd가 아님 → `systemctl` 사용 불가	setup.sh에 `service` 명령 대체 경로 추가. 별도로 systemd가 도는 Ubuntu 24.04 컨테이너를 만들어 실제 VM과 같은 경로로 재검증
테스트 컨테이너에서 pip 설치 실패	AI 환경의 보안 프록시가 TLS 인증서를 바꿔 끼워 검증 실패 (AI 환경에만 있는 문제)	호스트에서 패키지 파일을 받아 오프라인 설치로 테스트. 학생 VM에는 해당 없음
Rocky Linux 9 검증 불가	같은 TLS 문제로 dnf 설치 실패	RHEL 계열 분기(dnf·firewalld·SELinux)는 코드만 작성하고 **미검증**으로 명시

3. 예상 질문과 실제 질문 비교

15개
사전 예상 질문

1개
실제로 입력한 질문 (같은 내용 재전송 1회)

7 + 3
AI가 스스로 나눈 하위 작업 + 도중에 해결한 문제

#	예상 질문	직접 질문?	실제로 어떻게 해결됐나
1	VM 설치 사양·네트워크	안 함	**미해결** — AI는 VM이 이미 있다고 보고 다루지 않음. VM 설치는 직접 해야 함
2	3-Tier 개념, 계층별 역할	안 함	A-1 구성표로 설명됨
3	한 VM에 구성해도 되나, 소프트웨어 선택	안 함	A-1: AI가 Nginx/Flask/MariaDB 선택 (선택 이유도 제시)
4	Nginx 설치·실행	안 함	setup.sh 1·5단계
5	WAS 선택·설치	안 함	A-4 (Tomcat 대신 Gunicorn+Flask)
6	MariaDB 설치, root 비밀번호, DB·계정	안 함	A-2 — root 비밀번호 대신 Ubuntu 기본 소켓 인증 사용, 앱 전용 계정 별도 생성
7	Nginx - WAS 연동	안 함	A-5
8	로그인 페이지·처리 코드	안 함	A-3
9	테이블 설계, 비밀번호 저장	안 함	A-2, A-3 (scrypt 해시)
10	세션/쿠키	안 함	A-3
11	WAS→DB 접속 코드	안 함	A-3 (PyMySQL)
12	호스트→VM 네트워크	안 함	**부분** — A-6 설정 방법은 제시됐지만 AI는 내 하이퍼바이저를 직접 확인할 수 없어 내가 적용·확인해야 함
13	방화벽	안 함	A-6 (setup.sh 6단계)
14	502 Bad Gateway 해결	안 함	A-5 안내 페이지 + README 트러블슈팅 표
15	재부팅 후 자동 실행	안 함	A-4 (systemd)

예상 리스트에 없었는데 AI가 다룬 것

- **보안**: SQL Injection 방지, CSRF 토큰, ID/PW 중 무엇이 틀렸는지 숨기기(응답 시간까지 맞춤), 로그인 시 세션 재발급, 로그인 요청 수 제한(429), 보안 헤더
- **계층 분리**: WAS·DB를 `127.0.0.1` 에만 바인딩, DB 계정 최소 권한, 서비스를 root가 아닌 전용 계정으로 실행
- **환경 함정**: MariaDB `localhost` / `127.0.0.1` 계정 구분, 80 포트를 apache2가 이미 쓰는 경우, Rocky의 SELinux 차단, 로케일에 따라 명령 출력 문구가 달라질 수 있는 점(`LC_ALL=C` 로 고정)

비교 결과 분석

- **질문 수**: 15개를 예상했지만 실제로는 1개. 에이전트형 도구는 큰 문제 하나를 주면 스스로 쪼개서 처리하기 때문에, "어떻게 하나요?" 식의 질문을 여러 번 할 필요가 없었다.
- **질문 내용**: 예상 질문은 대부분 "설치 방법" 수준이었는데, AI는 보안·운영까지 다뤘다. 반대로 **VM 설치(1번)와 하이퍼바이저 네트워크(12번)**처럼 AI가 직접 볼 수 없는 내 PC 환경은 끝까지 내 몫으로 남았다.
- **질문의 성격 변화**: 할 일이 "어떻게 만들지 묻기"에서 "AI가 만든 것을 읽고 이해하고 내 환경에서 확인하기"로 바뀌었다.

4. AI 도구를 사용하면서 느낀 점

장점

1. **속도** — 질문 한 번으로 설계, 코드, 설정 파일, 설치 스크립트, 테스트까지 나왔다. 혼자 했다면 설치 방법을 찾는 데만 며칠이 걸렸을 것이다.
2. **몰랐던 것까지 챙김** — CSRF, 세션 재발급, DB 최소 권한처럼 예상 질문에 없던 보안·운영 요소가 들어갔다. "무엇을 물어야 하는지 모르는 것"을 채워 줬다.
3. **직접 실행해서 검증** — 답만 주는 게 아니라 설치하고 테스트를 돌려 28개 항목 통과를 확인했고, 일부러 WAS·DB를 꺼서 오류 상황도 확인했다.
4. **문서화** — README, 트러블슈팅 표, 코드 주석이 함께 만들어져 나중에 다시 보기 쉽다.

단점·한계

1. **AI 환경 ≠ 내 VM** — AI는 클라우드 컨테이너에서 검증했기 때문에 VirtualBox 네트워크 설정이나 내 VM의 방화벽은 확인하지 못했다. 마지막 확인은 결국 사람이 해야 한다.
2. **이해 없이 결과만 얻을 위험** — 너무 빨리 끝나서, 코드를 직접 읽지 않으면 무엇을 왜 했는지 모른 채 제출하게 된다. 배우려면 결과물을 읽고 추가 질문을 해야 했다.
3. **AI가 선택을 대신함** — 기술 스택(Flask)을 AI가 정했다. 수업에서 배운 스택(예: Apache·Tomcat)이 있다면 질문에 조건을 분명히 적어야 한다.
4. **검토할 양이 많음** — 결과물이 많아 어디가 핵심인지 고르는 데 시간이 걸렸고, 데모 계정 비밀번호 같은 기본값은 사람이 판단해서 바꿔야 한다.

> **정리** — AI 도구는 "답을 알려주는 검색창"이라기보다 "같이 일하는 동료"에 가까웠다. 반복적이고 시간이 드는 작업은 AI에게 맡기고, **요구사항을 정확히 전달하는 것, 내 환경에서 검증하는 것, 결과를 이해하는 것**은 사람이 맡아야 한다는 것을 느꼈다.

부록. 실행 화면과 테스트 결과

아래 화면은 AI 검증 환경(systemd가 동작하는 Ubuntu 24.04 컨테이너를 VM으로 보고, 바깥에서 VM IP `172.17.0.2` 로 접속)에서 촬영했다. 본인 VM에서 호스트 브라우저로 접속한 화면(주소창 포함)을 추가하거나 교체할 것.

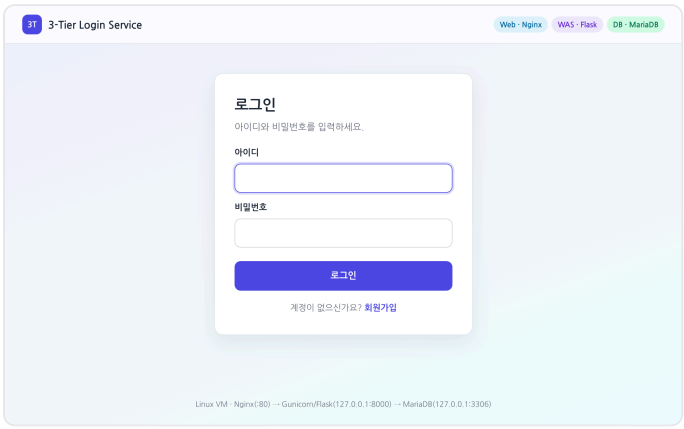


그림 2. 로그인 화면 (CSS는 Nginx가 직접 제공)

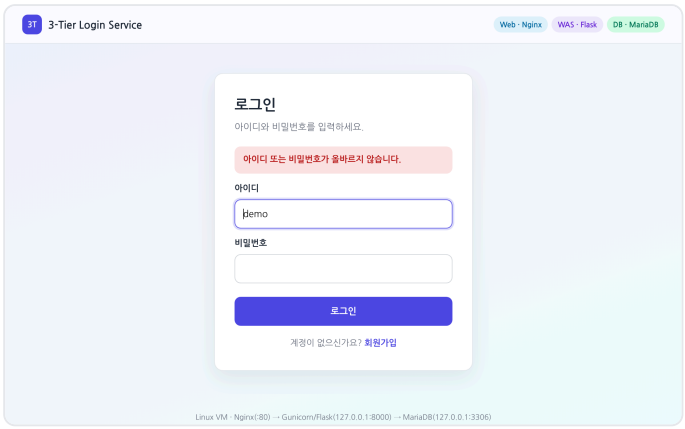


그림 3. 틀린 비밀번호 → 401, ID/PW 구분 없는 메시지

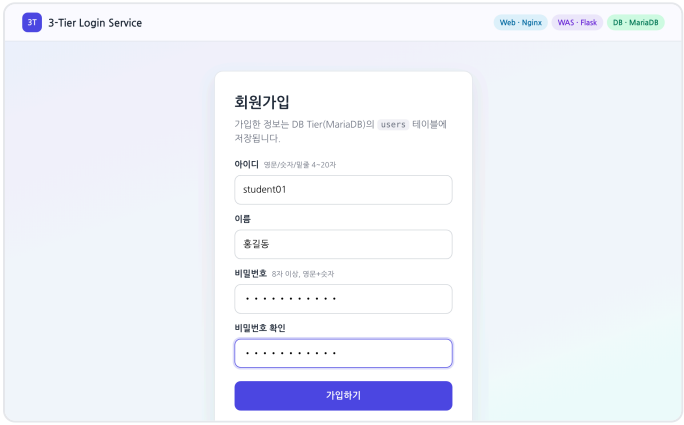


그림 4. 회원가입 (users 테이블에 해시로 저장)

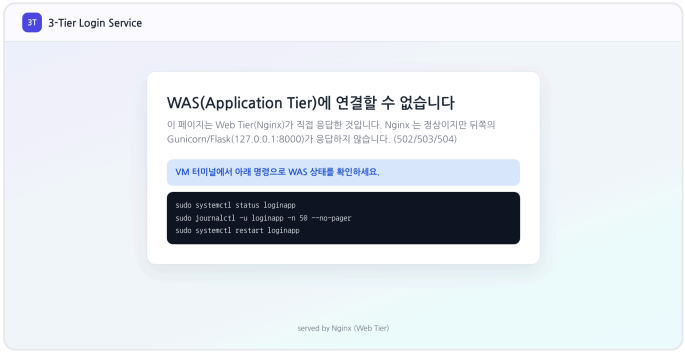


그림 5. WAS를 끈 상태 → Nginx가 직접 502 안내

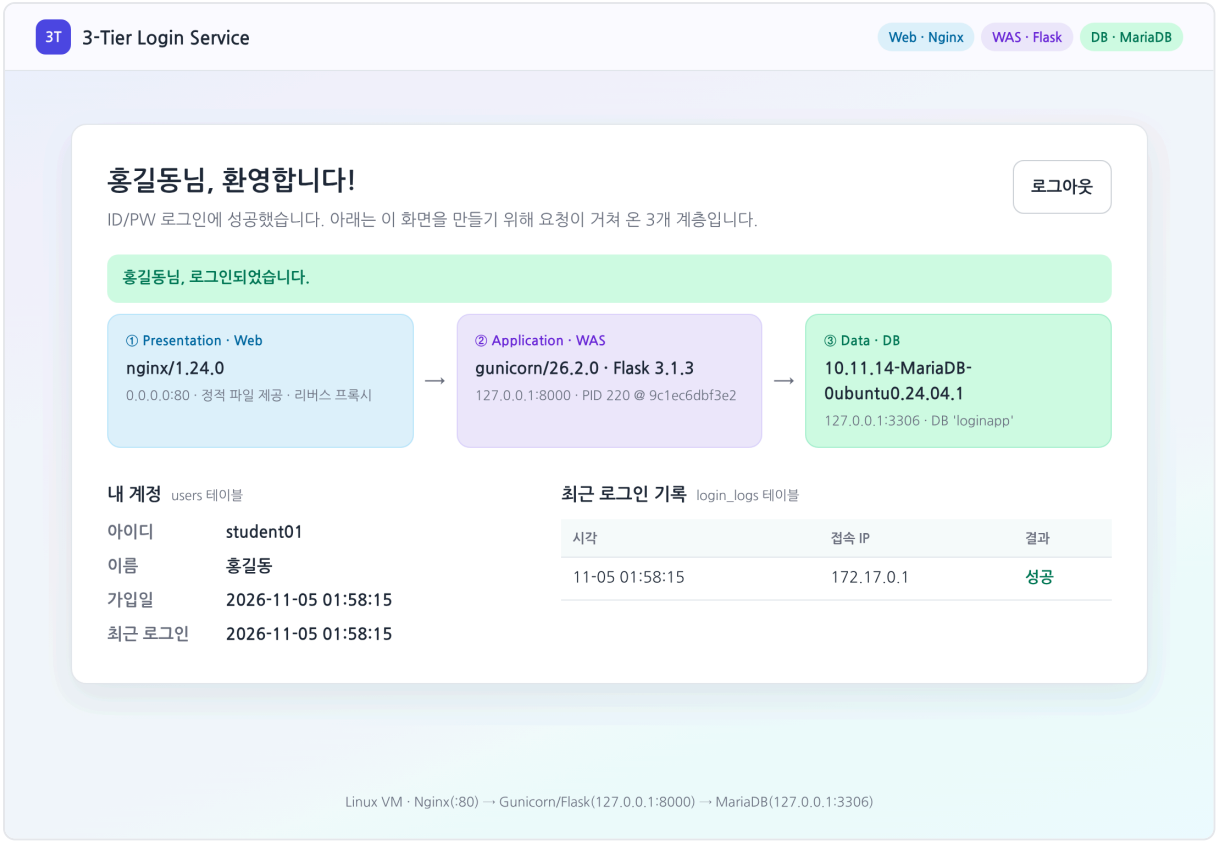


그림 6. 로그인 성공 — 요청이 거친 3계층(nginx → gunicorn/Flask → MariaDB)과 DB에서 읽은 로그인 기록

E2E 테스트 결과 (`sudo bash tests/e2e_test.sh` , VM 내부)

```
[1] Web Tier (Nginx)
  PASS GET /health → 200                          PASS 헬스체크: WAS · DB 정상 (db=ok)
  PASS 요청이 Nginx 를 거쳐 WAS 로 전달됨          PASS 정적 파일 /static/style.css → 200
  PASS 정적 파일에 Nginx 캐시 헤더                 PASS 보안 헤더 X-Frame-Options: DENY
[2] WAS Tier (Flask) : 접근 제어 / CSRF
  PASS 비로그인 GET / → /login                     PASS 비로그인 GET /home → /login (접근 차단)
  PASS GET /login → 200, 로그인 폼 + CSRF 토큰     PASS CSRF 토큰 없이 POST /login → 400 거부
[3] WAS + DB Tier : 로그인 / 회원가입
  PASS 틀린 비밀번호 → 401 + 실패 메시지           PASS 없는 아이디 → 401 + 같은 메시지
  PASS SQL Injection 시도 (' OR '1'='1' --) → 401  PASS 회원가입: 약한 비밀번호 → 400
  PASS 회원가입: 신규 계정 → 302 /login            PASS 회원가입: 중복 아이디 → 409
  PASS 신규 계정 로그인 성공 → 302 /home           PASS 세션 쿠키에 HttpOnly 속성
  PASS GET /home → 200, 이름 + 로그인 기록         PASS 메인 화면에 3계층 정보
  PASS 로그아웃 → 302 /login                       PASS 로그아웃 후 /home → /login (세션 삭제)
  PASS 데모 계정(demo) 로그인 → 302 /home
[4] 계층 분리 / 저장 방식
  PASS Web 80 포트는 0.0.0.0 에 공개               PASS WAS 8000 포트는 127.0.0.1 에만 바인딩
  PASS DB 3306 포트는 127.0.0.1 에만 바인딩        PASS DB 에 비밀번호가 해시(scrypt:…)로 저장됨
  PASS 로그인 시도가 login_logs 테이블에 기록됨

결과: 28/28 통과
```

```
# VM 바깥(호스트 역할)에서 직접 접근 시도
$ curl http://172.17.0.2:8000/   →  Failed to connect ... port 8000   # WAS 직접 접근 차단
$ curl telnet://172.17.0.2:3306  →  Failed to connect ... port 3306   # DB 직접 접근 차단
# 재시작 후
$ systemctl is-active nginx mariadb loginapp  →  active active active
```